SOFTWARE REFERENCE MANUAL

SYNERGY ENGINEERING LABS

Aegis PQCore

Foundational Post-Quantum Cryptography Engine

MODULE	VERSION	NIST ALGORITHMS
aegis-pqcore	1.0 — Production	6 Families / All Levels

STATUS

✓ Verified — 1827 Tests Passing

NAVIGATION

Table of Contents

1 About Aegis §1

2 Module Overview — aegis-pqcore §2

3 Technical Architecture §3

4 Supported Algorithms & Security Levels §4

5 PQC Integration Without Rebuilding §5

6 API Reference & Usage §6

7 Validation, Testing & KAT Framework §7

8 Security Architecture & Compliance §8

SECTION 1

About Aegis

The Quantum-Safe Cryptography Platform

Aegis is Synergy Engineering Labs' patent-pending post-quantum cryptography platform — a comprehensive, production-grade suite that delivers NIST-standardized quantum-safe algorithms across every major computing environment without requiring you to rebuild your systems from the ground up.

The global transition to post-quantum cryptography is no longer a future concern — it is an immediate engineering imperative. Quantum computers capable of breaking RSA, ECDH, and ECDSA are advancing on a timeline that demands action today. Aegis was engineered to solve the hardest part of that transition: **getting quantum-safe cryptography into production systems without disrupting existing architectures**.

Aegis implements all six NIST Post-Quantum Cryptography finalized and selected algorithms across the full spectrum of computing environments — from bare-metal microcontrollers and wearable devices to cloud-native serverless functions, blockchain virtual machines, and browser-based WebAssembly runtimes. Each module is independently deliverable, self-contained, and designed to slot into existing systems as a cryptographic layer upgrade rather than a wholesale rewrite.

Why Aegis

NIST Compliant

Drop-In Integration

All algorithms conform to FIPS 203, 204, 205, 206, 207, and 208 standards. Known-Answer Test vectors validated against official NIST test vectors.

Aegis modules expose clean APIs that mirror existing cryptographic conventions, enabling integration without architectural rebuilds.

15 Platform Modules

Rust, Python, Node.js, WebAssembly, WASI, iOS/Android, embedded bare-metal, hardware HSM/FPGA, blockchain VMs, and more.

Production Hardened

Constant-time implementations, automatic memory zeroization, fault-injection tested, and strict Rust quality gates enforced across all modules.

Audit Ready

Full SBOM generation, provenance artifacts, frozen API surfaces, CMVP/ACVP compliance documentation, and complete audit binders.

Key Lifecycle Management

Built-in key rotation, usage tracking, Merkleized audit logs, and cryptographic proof of destruction — managed through the KLC-RB subsystem.

The Aegis Module Ecosystem

Aegis is organized as 15 independent, self-contained modules — each targeting a specific platform or integration context. Every module shares the same algorithmic core (`aegis_crypto_clean`) but is packaged, compiled, and exposed through APIs optimized for its target environment. You license only the modules you need.

Module	Target Environment	Primary Language
`aegis-pqcore`	Foundational C/Rust algorithm corpus	C + Rust
`aegis-pqrust`	Native Rust applications	Rust
`aegis-pqwasm`	Browser / WebAssembly	JavaScript / WASM
`aegis-pqnodejs`	Node.js server-side	JavaScript / WASM
`aegis-pqpython`	Python applications	Python (FFI)

`aegis-pqvm`	Blockchain / Smart Contract VMs	Rust / EVM ABI
`aegis-pqwasi`	Edge computing / serverless	WASI / Rust
`aegis-pqnist`	NIST reference + Rust FFI bindings	Rust FFI / C
`aegis-pqnostd`	Embedded / bare-metal (no_std)	Rust (no_std)
`aegis-pqmobile`	iOS & Android	Rust + FFI
`aegis-pqwear`	Wearable devices	Rust
`aegis-pqsmart`	Smart devices / IoT	Rust
`aegis-pqhw`	Hardware (FPGA / ASIC / HSM)	Rust + C
`aegis-pqmavr`	Spatial computing / constrained edge (AR/VR)	Rust + AVR C
`aegis-pqsynq`	SynQ VM & compiler integration	Rust

SECTION 2

Module Overview — aegis-pqcore

The Foundation Layer

`aegis-pqcore` is the foundational Aegis module — the authoritative C algorithm corpus and Rust runtime that every other Aegis module ultimately builds upon. It is the single source of cryptographic truth for the entire Aegis ecosystem.

`aegis-pqcore` (also referred to internally as `aegis-pqclean`) operates on two parallel production tracks that work in concert:

Track 1 — Clean C Algorithm Corpus

The first track is a PQClean-style clean C implementation of all six NIST PQC algorithm families. These implementations follow strict portability and auditability standards: no platform-specific intrinsics, no unsafe C practices, no undefined behavior. The corpus is organized under `crypto_kem/` (key encapsulation mechanisms) and `crypto_sign/` (digital signature schemes), with each algorithm variant fully isolated in its own directory tree.

This clean C corpus serves multiple purposes: it provides the most auditable, readable reference implementation for security review; it supplies the cryptographic source that all Rust wrappers compile against; and it is the basis for Known-Answer Test (KAT) validation against official NIST test vectors.

Track 2 — Rust Key Lifecycle and Beacon Runtime (klc-rb)

The second track is a Rust-native runtime crate (`klc-rb`) that provides key lifecycle management and quantum randomness beacon functionality. This runtime layer handles the operational security concerns that raw algorithm implementations don't address: key generation tracking, rotation policies, usage enforcement, cryptographic proof of key destruction, Merkleized audit logs, and entropy chain management.

What aegis-pqcore Is For

Reference Source

The canonical C implementation that all other modules link against or derive from. When a module needs verified PQC source, it points here.

Validation Harness

Hosts the NIST KAT test framework. All 1,827 tests pass, 270 are configuration-skipped. Zero failures.

Key Lifecycle Runtime

The `klc-rb` crate provides lifecycle management: key creation, rotation, expiry, usage tracking, and audit-log integrity.

Quantum Randomness Beacon

ML-KEM-based entropy generation with ML-DSA signature proofs and beacon chaining for verifiable randomness.

SECTION 3

Technical Architecture

Directory Structure

```
aegis-pqcore/
├── crypto_kem/              # KEM algorithm implementations (C)
│   ├── ml-kem-512/
│   ├── ml-kem-768/
│   ├── ml-kem-1024/
│   ├── hqc-kem-128/
│   ├── hqc-kem-192/
│   ├── hqc-kem-256/
│   └── cmce/               # Classic McEliece variants
├── crypto_sign/             # Signature algorithm implementations (C)
│   ├── ml-dsa-44/
│   ├── ml-dsa-65/
│   ├── ml-dsa-87/
│   ├── fn-dsa-512/
│   ├── fn-dsa-1024/
│   └── slh-dsa-*/          # 12 SLH-DSA variants (SHA2 + SHAKE)
├── common/                  # Shared C utilities (randombytes, fips202, etc.)
├── klc-rb/                  # Rust key lifecycle & beacon runtime crate
│   └── src/
│       ├── lifecycle.rs     # Key lifecycle manager
│       ├── beacon.rs        # Quantum randomness beacon
│       └── lib.rs
├── test/                    # Python KAT test harness (pytest)
├── scripts/                 # CI/CD, policy, SBOM, provenance scripts
├── docs/                    # Security, compliance, release documentation
└── artifacts/               # Generated evidence and release outputs
```

Core Design Principles

1 **Source Isolation**

Each algorithm variant lives in its own directory with no shared mutable state. Cross-variant contamination is structurally impossible at the filesystem level.

2 **PQClean Compatibility**

All C implementations conform to PQClean API conventions: `crypto_kem_keypair`, `crypto_kem_enc`, `crypto_kem_dec`, `crypto_sign_keypair`, `crypto_sign`, `crypto_sign_open`. This maximizes auditability and compatibility with existing tooling.

3 **Deterministic Source Pinning**

The `AEGIS_PQ_SOURCE_ROOT` environment variable allows explicit override of the cryptographic C source tree at build time, ensuring reproducible and auditable builds.

4 **Namespace Isolation**

Each algorithm variant uses a unique namespace prefix (e.g., `PQCLEAN_MLDSA44_CLEAN_`) to prevent symbol collisions when multiple variants are linked together.

The klc-rb Runtime Subsystem

The `klc-rb` (Key Lifecycle and Randomness Beacon) Rust crate is the operational security layer of `aegis-pqcore`. It provides:

Key Lifecycle Manager

A fixed-capacity lifecycle manager that enforces cryptographic key hygiene throughout a key's operational lifetime. It tracks key generation events, enforces rotation intervals, records usage counts, implements expiry policies, and generates cryptographic proofs of key destruction using Merkleized audit logs. The design is compatible with `no_std` embedded environments when appropriate feature flags are enabled.

Quantum Randomness Beacon

The beacon subsystem uses ML-KEM encapsulation to generate unpredictable entropy, signs each beacon output with ML-DSA, and chains beacon outputs together so that any tampering with historical beacon values is detectable. This provides a verifiable, post-quantum-secure source of randomness suitable for key generation, nonce derivation, and protocol challenges.

SECTION 4

Supported Algorithms & Security Levels

Algorithm	Type	Variants	NIST Standard	Status
ML-KEM Module-Lattice KEM	KEM	ML-KEM-512, ML-KEM-768, ML-KEM-1024	FIPS 203	✓ Complete
ML-DSA Module-Lattice DSA	Signature	ML-DSA-44, ML-DSA-65, ML-DSA-87	FIPS 204	✓ Complete
FN-DSA Falcon / Hash-based DSA	Signature	FN-DSA-512, FN-DSA-1024	FIPS 206	✓ Complete
SLH-DSA Stateless Hash-based DSA	Signature	12 variants: SHA2 128/192/256 fast/small, SHAKE 128/192/256 fast/small	FIPS 205	✓ Complete
HQC-KEM Hamming Quasi-Cyclic KEM	KEM	HQC-KEM-128, HQC-KEM-192, HQC-KEM-256	FIPS 207	✓ Complete
CMCE-KEM Classic McEliece	KEM	348864, 460896, 6688128 (feature-gated)	FIPS 208	⚠ Experimental

ℹ️ **Security Level Guidance**

ML-KEM-768 and ML-DSA-65 provide NIST Security Level 3, equivalent to AES-192. For most enterprise and government applications, these are the recommended default variants. ML-KEM-1024 and ML-DSA-87 provide Level 5 (AES-256 equivalent) for the most sensitive applications.

⚠️ **Classic McEliece Notice**

CMCE-KEM is feature-gated and disabled by default. Its key sizes are extremely large (hundreds of kilobytes for public keys), making it unsuitable for most network protocols. Enable only when large key sizes are acceptable and maximum security margin is required.

SECTION 5

PQC Integration Without Rebuilding

A fundamental design goal of Aegis is to enable post-quantum cryptography adoption without requiring organizations to rebuild their existing systems from scratch. `aegis-pqcore` achieves this through several integration patterns.

Pattern 1 — Hybrid Cryptography (Recommended)

The most common and safest integration pattern runs classical and post-quantum algorithms in parallel. A session key is established by performing both a classical ECDH exchange and an ML-KEM encapsulation; the resulting shared secrets are combined using a KDF. This means your system remains secure against both classical and quantum adversaries, and you can roll back the PQC layer if needed without service interruption.

```
// Example: Hybrid TLS-style key establishment
// Step 1: Classical ECDH (existing code unchanged)
let classical_secret = ecdh_exchange(&peer_public_key)?;

// Step 2: ML-KEM encapsulation (new — aegis-pqcore)
let (ciphertext, pqc_secret) = mlkem768_encapsulate(&pq_public_key)?;

// Step 3: Combine via KDF (minimal new code)
let session_key = hkdf_extract(&classical_secret, &pqc_secret);
```

Pattern 2 — Algorithm Substitution

For systems that can tolerate a protocol upgrade, Aegis supports direct algorithm substitution. The C API surface of `aegis-pqcore` mirrors the PQClean convention, so existing code that calls `crypto_kem_keypair` can be relinked against an Aegis ML-KEM implementation with minimal source changes.

Pattern 3 — Signature Layer Upgrade

For document signing, code signing, or authentication tokens, an existing ECDSA or RSA signature can be accompanied by an additional ML-DSA or FN-DSA signature. Verification code checks both

signatures. Over time the classical signature can be phased out.

Getting Started with aegis-pqcore

1. **Install Prerequisites**

   Rust 1.70+, Python 3.9+ (for KAT tests), a C11-capable compiler (GCC or Clang), and Make.

2. **Run the Security Baseline**

   `cd aegis-pqcore && ./scripts/ci_security_baseline.sh` — validates the full C build, runs Rust quality gates, and verifies naming policy compliance.

3. **Run KAT Validation**

   `./scripts/run_deterministic_kat.sh` — executes the Known-Answer Test harness against official NIST test vectors. All 1,827 passing.

4. **Link Against Your Project**

   Reference `aegis-pqcore` as the `AEGIS_PQ_SOURCE_ROOT` in your Rust build scripts, or link the compiled C objects directly into your C/C++ project.

SECTION 6

API Reference & Usage

C API — Key Encapsulation (ML-KEM)

```
/* ML-KEM-768 — FIPS 203, Security Level 3 */
/* Key generation */
int PQCLEAN_MLKEM768_CLEAN_crypto_kem_keypair(
    uint8_t *pk,   /* public key: 1184 bytes */
    uint8_t *sk    /* secret key: 2400 bytes */
);

/* Encapsulation */
int PQCLEAN_MLKEM768_CLEAN_crypto_kem_enc(
```

```
    uint8_t *ct,   /* ciphertext: 1088 bytes */
    uint8_t *ss,   /* shared secret: 32 bytes */
    const uint8_t *pk  /* recipient public key */
);

/* Decapsulation */
int PQCLEAN_MLKEM768_CLEAN_crypto_kem_dec(
    uint8_t *ss,          /* shared secret: 32 bytes */
    const uint8_t *ct,    /* ciphertext */
    const uint8_t *sk     /* secret key */
);
```

C API — Digital Signatures (ML-DSA)

```
/* ML-DSA-65 — FIPS 204, Security Level 3 */
int PQCLEAN_MLDSA65_CLEAN_crypto_sign_keypair(
    uint8_t *pk,  /* public key: 1952 bytes */
    uint8_t *sk   /* secret key: 4032 bytes */
);

int PQCLEAN_MLDSA65_CLEAN_crypto_sign_signature(
    uint8_t *sig,         /* signature: 3309 bytes */
    size_t  *siglen,
    const uint8_t *m,     /* message */
    size_t   mlen,
    const uint8_t *sk     /* signing key */
);

int PQCLEAN_MLDSA65_CLEAN_crypto_sign_verify(
    const uint8_t *sig,
    size_t  siglen,
    const uint8_t *m,
    size_t  mlen,
    const uint8_t *pk
);
```

Rust — klc-rb Key Lifecycle

```
use aegis_klc_rb::{KeyLifecycleManager, RotationPolicy};

// Initialize with a 32-key capacity and 30-day rotation
let mut mgr = KeyLifecycleManager::new(
    RotationPolicy::interval_days(30),
    /* capacity: */ 32,
```

```
)?;

// Generate a tracked key
let key_id = mgr.generate_key("ml-kem-768")?;

// Retrieve active key
let key = mgr.active_key(&key_id)?;

// Rotate when policy triggers
if mgr.rotation_due(&key_id) {
    mgr.rotate_key(&key_id)?;
}

// Destroy with cryptographic proof
let proof = mgr.destroy_key(&key_id)?;
// proof contains Merkle-path audit evidence
```

Key Size Reference

Algorithm	Public Key	Secret Key	Ciphertext / Signature
ML-KEM-512	800 B	1632 B	768 B (ct)
ML-KEM-768	1184 B	2400 B	1088 B (ct)
ML-KEM-1024	1568 B	3168 B	1568 B (ct)
ML-DSA-44	1312 B	2528 B	2420 B (sig)
ML-DSA-65	1952 B	4032 B	3309 B (sig)
ML-DSA-87	2592 B	4896 B	4627 B (sig)
FN-DSA-512	897 B	1281 B	~666 B (sig, variable)
FN-DSA-1024	1793 B	2305 B	~1280 B (sig, variable)

SECTION 7

Validation, Testing & KAT Framework

✅ **Verification Status: PASSING**

Full pytest run:

1827 passed, 270 skipped, 0 failed

(February 2026 close-out verification)

Known-Answer Test (KAT) Framework

The KAT framework is implemented in Python (pytest) under `aegis-pqcore/test/`. It exercises each algorithm variant against official NIST test vectors from `5-nist-kat-vectors/`, running deterministic key generation, encapsulation/decapsulation, signing, and verification sequences whose outputs are compared byte-for-byte against the NIST reference values.

```
# Run full KAT suite
cd aegis-pqcore
pytest -x -q

# Run deterministic replay
./scripts/run_deterministic_kat.sh

# Run validation harness
./scripts/run_validation_harness.sh

# Replay saved validation corpus
./scripts/replay_validation_harness.sh
```

Quality Gates

```
# Full security baseline
./scripts/ci_security_baseline.sh

# Naming policy compliance
./scripts/check_naming_policy.sh

# Verify no KAT tests are marked ignore
./scripts/check_no_ignored_kat.sh
```

```
# Release policy gate
./scripts/check_release_policy.sh
```

Fault Injection Testing

The broader Aegis test suite (under `4-known-answer-tests/`) includes adversarial and fault injection scenarios: signature tampering, message modification, key corruption, replay attacks, and timing side-channel resistance checks. All fault injection tests pass for the algorithms hosted in `aegis-pqcore`.

SECTION 8

Security Architecture & Compliance

Constant-Time Implementation

All secret-dependent operations within the C algorithm corpus are implemented in constant time. Branching on secret values, table lookups indexed by secret data, and any other variable-timing patterns that could leak key material through timing side channels are prohibited. This is enforced through code review and static analysis.

Memory Zeroization

Secret keys, intermediate computations, and shared secrets are zeroized from memory immediately after use. The Rust runtime layer uses the `zeroize` crate to ensure that secret material is overwritten before deallocation, preventing extraction from heap dumps or core files.

FIPS 203/204/205/206/207 Compliance

All algorithm implementations are validated against NIST FIPS standards. The KAT test framework specifically validates the deterministic output consistency required by these standards. CMVP boundary documentation is provided in `docs/compliance/CMVP_BOUNDARY_AND_SELFTEST.md`, and an ACVP harness plan is available at `docs/compliance/ACVP_HARNESS.md`.

Threat Model Coverage

- Quantum adversaries (Shor/Grover)
- Side-channel timing attacks
- Fault injection
- Key material extraction
- Replay attacks
- Signature forgery
- Ciphertext malleability
- Namespace collision

Reporting Vulnerabilities

Security vulnerabilities should be reported privately to **security@aegis-crypto.org**. Do not create public issue reports for security vulnerabilities. Response SLA: 24-hour initial acknowledgement, 72-hour status update, 30-day resolution target.

SECTION 9

Operations, Release & Distribution

⚠️ **Distribution Policy**

Aegis is not available via public package managers (npm, crates.io, PyPI, etc.). Distribution is exclusively through licensed channels controlled by Synergy Engineering Labs. All Aegis code is 100% production-ready — no placeholder or stub implementations are permitted.

Release Operations

```
# Generate SBOM (Software Bill of Materials)
./scripts/generate_sbom.sh

# Generate release manifest
./scripts/generate_release_manifest.sh

# Generate cryptographic provenance
```

```
./scripts/generate_provenance.sh

# Package customer bundle
./scripts/package_customer_bundle.sh

# Verify release bundle integrity
./scripts/verify_release_bundle.sh
```

CI/CD Workflows

Module-local GitHub Actions workflows are defined in `.github/workflows/`:

- ci.yml — Main quality gates
- module-security-baseline.yml
- fuzz-ci.yml — Fuzz testing
- kat-cross-platform.yml
- release-provenance.yml

Documentation Index

Document	Path
User Manual	`docs/manual/USER_MANUAL.md`
Threat Model	`docs/security/THREAT_MODEL.md`
Security Case Bundle	`docs/security/SECURITY_CASE_BUNDLE.md`
ACVP Harness Plan	`docs/compliance/ACVP_HARNESS.md`
CMVP Boundary	`docs/compliance/CMVP_BOUNDARY_AND_SELFTEST.md`
Release Policy	`docs/release/RELEASE_POLICY.md`
Production Checklist	`PRODUCTION_CHECKLIST.md`